

Sentiment Analysis: Judge Emotion About Brands and Products

Author: Joel Muoki Andrew

Date: June 2026

PROJECT OVERVIEW

This notebook performs **sentiment analysis on tweets** directed at brands and products using the [CrowdFlower dataset](#) (9,093 tweets, sourced August 2013).

1. Business Understanding

1.1 Problem Statement

Social media platforms generate millions of brand-related opinions daily. Manually tracking public sentiment at scale is infeasible. The goal is to **automatically classify the emotion expressed in a tweet** toward a brand or product as:

- **Positive**
- **Negative**
- **No emotion**
- (Possibly: "I can't tell")

1.2 Business Objective

Build a machine learning classifier that can accurately detect the **sentiment** of a tweet directed at a brand/product, enabling brands to monitor public perception in real time.

1.3 Business Questions

1. Can we automatically detect whether a tweet expresses emotion toward a brand?
 2. Which sentiment class is the hardest for the model to predict correctly?
 3. Between Naive Bayes, Logistic Regression and tuned Logistic Regression which model is better suited for brand sentiment classification?
-

2. Data Understanding

2.1 Imports & Setup

```
In [68]: # Loading Libraries
import pandas as pd
import numpy as np
import re
import string
import matplotlib.pyplot as plt
import seaborn as sns

# NLP Libraries
import nltk
from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer
from sklearn.model_selection import train_test_split
from sklearn.metrics import f1_score, accuracy_score, recall_score
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score
import pandas as pd
import re
import string

# Download NLTK data (run once)
nltk.download('stopwords')
nltk.download('wordnet')
nltk.download('punkt')
```

```
[nltk_data] Downloading package stopwords to
[nltk_data] C:\Users\Joel\AppData\Roaming\nltk_data...
[nltk_data] Package stopwords is already up-to-date!
[nltk_data] Downloading package wordnet to
[nltk_data] C:\Users\Joel\AppData\Roaming\nltk_data...
[nltk_data] Package wordnet is already up-to-date!
[nltk_data] Downloading package punkt to
[nltk_data] C:\Users\Joel\AppData\Roaming\nltk_data...
[nltk_data] Package punkt is already up-to-date!
```

Out[68]: True

2.2 Loading Dataset

```
In [69]: # Loading our dataset
df = pd.read_csv('judge-1377884607_tweet_product_company.csv', encoding='latin1')
df.head()
```

Out[69]:

	tweet_text	emotion_in_tweet_is_directed_at	is_there_an_emotion_directed_at_a_brand_or
0	.@wesley83 I have a 3G iPhone. After 3 hrs twe...	iPhone	Negative
1	@jessedee Know about @fludapp ? Awesome iPad/i...	iPad or iPhone App	Positive
2	@swonderlin Can not wait for #iPad 2 also. The...	iPad	Positive
3	@sxsw I hope this year's festival isn't as cra...	iPad or iPhone App	Negative
4	@sxtxstate great stuff on Fri #SXSW: Marissa M...	Google	Positive

2.3 Data Inspection

```
In [70]: # Checking dataset shape
df.shape
```

```
Out[70]: (9093, 3)
```

```
In [71]: # Checking column names
df.columns
```

```
Out[71]: Index(['tweet_text', 'emotion_in_tweet_is_directed_at',
               'is_there_an_emotion_directed_at_a_brand_or_product'],
              dtype='object')
```

```
In [72]: # Renaming our columns
df = df.rename(columns={
    'tweet_text': 'Tweet',
    'emotion_in_tweet_is_directed_at': 'Target',
    'is_there_an_emotion_directed_at_a_brand_or_product': 'Sentiment'
})
```

```
In [73]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 9093 entries, 0 to 9092
Data columns (total 3 columns):
#   Column      Non-Null Count  Dtype
---  -
0   Tweet       9092 non-null   object
1   Target      3291 non-null   object
2   Sentiment    9093 non-null   object
dtypes: object(3)
memory usage: 213.2+ KB
```

```
In [74]: # Missing values per column
print("Missing Values:")
print(df.isnull().sum())
print(f"\nTotal missing: {df.isnull().sum().sum()}")
```

Missing Values:

```
Tweet      1
Target    5802
Sentiment   0
dtype: int64
```

Total missing: 5803

```
In [75]: # Our data distribution(Tweet)
df['Tweet'].value_counts().head()
```

```
Out[75]: Tweet
RT @mention Marissa Mayer: Google Will Connect the Digital & Physical Worlds Through Mobile - {link} #sxsw      5
RT @mention Google to Launch Major New Social Network Called Circles, Possibly Today {link} #sxsw      4
RT @mention Marissa Mayer: Google Will Connect the Digital & Physical Worlds Through Mobile - {link} #SXSW      4
RT @mention Google to Launch Major New Social Network Called Circles, Possibly Today {link} #SXSW      3
RT @mention 🇺🇸 GO BEYOND BORDERS! 🇺🇸_ {link} 🇺🇸_ #edchat #musedchat #sxsw #sxswi #classical #newTwitter      2
Name: count, dtype: int64
```

```
In [76]: # Target distribution
df['Target'].value_counts()
```

```
Out[76]: Target
iPad      946
Apple     661
iPad or iPhone App  470
Google    430
iPhone    297
Other Google product or service  293
Android App      81
Android         78
Other Apple product or service  35
Name: count, dtype: int64
```

```
In [77]: # Sentiment Distribution
```

```
df['Sentiment'].value_counts()
```

```
Out[77]: Sentiment
No emotion toward brand or product    5389
Positive emotion                      2978
Negative emotion                      570
I can't tell                          156
Name: count, dtype: int64
```

```
In [78]: # Checking unique (Target)
df['Target'].unique()
```

```
Out[78]: array(['iPhone', 'iPad or iPhone App', 'iPad', 'Google', nan, 'Android',
               'Apple', 'Android App', 'Other Google product or service',
               'Other Apple product or service'], dtype=object)
```

```
In [79]: # Checking unique (Sentiment)
df['Sentiment'].unique()
```

```
Out[79]: array(['Negative emotion', 'Positive emotion',
               'No emotion toward brand or product', 'I can't tell'], dtype=object)
```

```
In [80]: # sample any 10 in our dataset(Tweet)
df['Tweet'].sample(10, random_state=42)
```

```
Out[80]: 5558    RT @mention best thing abt @mention for #SXSW?...
8350    Gary is starting a new show "Daily Grape"...
2303    The walk by Lady Bird Lake was lovely, but Goo...
927     C34: It's Not a Retreat! {link} [codes valid...
1963    Holler Gram for iPad on the iTunes App Store: ...
1672    I asked, "Why should(n't) I get an iPad 2...
4475    LOL, I don't see a single PC user around me...
4262    Video: Tim Wu explains to me #netneutrality, i...
8216    Rockin out at the Party Lake It's 1986 Party! ...
6936    RT @mention Will Google Circles take on Facebo...
Name: Tweet, dtype: object
```

```
In [81]: # Sample any 10 in our Dataset(Sentiment)
df['Sentiment'].sample(10, random_state=42)
```

```
Out[81]: 5558    No emotion toward brand or product
8350    No emotion toward brand or product
2303    Negative emotion
927     No emotion toward brand or product
1963    Positive emotion
1672    No emotion toward brand or product
4475    Positive emotion
4262    No emotion toward brand or product
8216    No emotion toward brand or product
6936    No emotion toward brand or product
Name: Sentiment, dtype: object
```

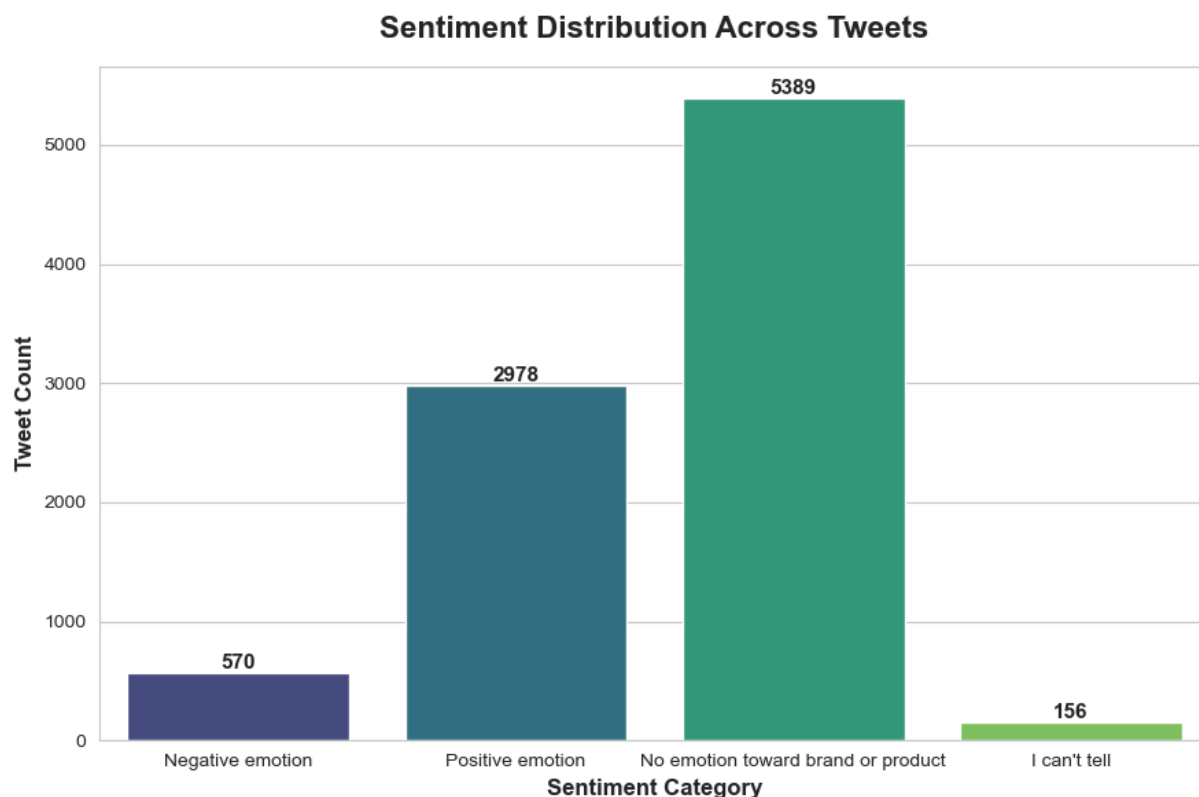
3. Exploratory Data Analysis

We explore distributions and relationships between columns before modeling.

3.1 Univariate Analysis

```
In [82]: # =====  
# Univariate Analysis: Sentiment Distribution  
# =====  
  
# Apply a visually appealing plotting style  
sns.set_style("whitegrid")  
  
# Create the figure  
plt.figure(figsize=(9, 6))  
  
# Visualize the frequency of each sentiment category  
ax = sns.countplot(  
    x='Sentiment',  
    data=df,  
    palette='viridis'  
)  
  
# Customize chart title and axis labels  
plt.title(  
    'Sentiment Distribution Across Tweets',  
    fontsize=16,  
    fontweight='bold',  
    pad=15  
)  
  
plt.xlabel(  
    'Sentiment Category',  
    fontsize=12,  
    fontweight='semibold'  
)  
  
plt.ylabel(  
    'Tweet Count',  
    fontsize=12,  
    fontweight='semibold'  
)  
  
# Display count labels above each bar  
for bar in ax.patches:  
    ax.annotate(  
        f'{int(bar.get_height())}',  
        (bar.get_x() + bar.get_width() / 2, bar.get_height()),  
        ha='center',  
        va='bottom',  
        fontsize=11,  
        fontweight='bold'  
    )  
  
# Improve spacing and presentation  
plt.tight_layout()
```

```
# Render the plot
plt.show()
```



Key Insight

The sentiment distribution shows that the majority of tweets (5,389) express no emotion toward the brand or product, indicating that most discussions are neutral or informational rather than opinion-based. Positive sentiment is the second most common category (2,978 tweets), suggesting that favorable opinions outweigh negative ones. Negative sentiment accounts for only 570 tweets, while a very small number of tweets (156) could not be classified. Overall, the dataset exhibits a predominantly neutral tone with considerably more positive than negative sentiment.

2.2 Bivariate Analysis

```
In [83]: # =====
# Bivariate Analysis: Sentiment by Target
# =====

# Create a larger figure for better readability
plt.figure(figsize=(14, 8))

# Plot sentiment frequencies grouped by target category
ax = sns.countplot(
    x='Sentiment',
    hue='Target',
    data=df,
    palette='mako'
```

```
)

# Add descriptive title and axis labels
plt.title(
    'Distribution of Sentiment Across Different Targets',
    fontsize=18,
    fontweight='bold',
    pad=15
)

plt.xlabel(
    'Sentiment Category',
    fontsize=13,
    fontweight='semibold'
)

plt.ylabel(
    'Number of Tweets',
    fontsize=13,
    fontweight='semibold'
)

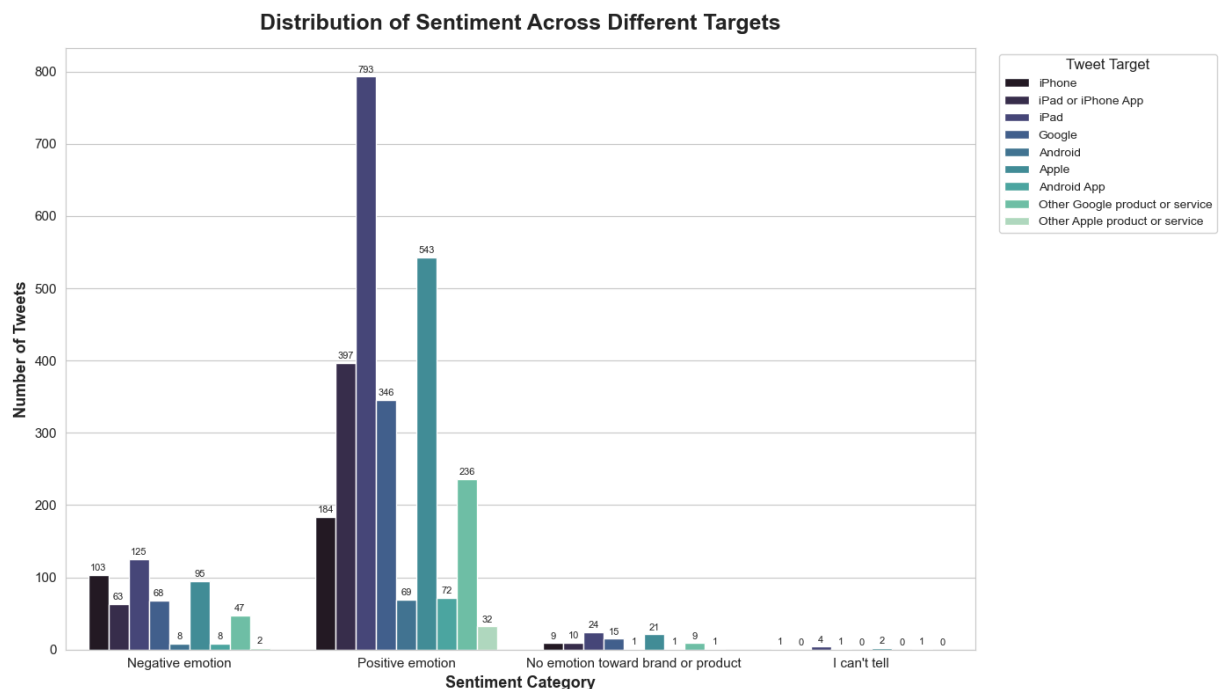
# Improve tick label appearance
plt.xticks(fontsize=11)
plt.yticks(fontsize=11)

# Display frequency labels on each bar
for container in ax.containers:
    ax.bar_label(
        container,
        fmt='%d',
        fontsize=8,
        padding=2
    )

# Position legend outside the plot area
plt.legend(
    title='Tweet Target',
    title_fontsize=12,
    fontsize=10,
    bbox_to_anchor=(1.02, 1),
    loc='upper left',
    frameon=True
)

# Adjust layout to prevent overlap
plt.tight_layout()

# Display the visualization
plt.show()
```

Key Insight

iPad/iPhone App dominates positive emotion (793 tweets) — more than double the next closest brand (Apple: 543). However, when you look at negative emotion, iPad/iPhone App (125) and iPhone (103) lead — meaning Apple's ecosystem attracts both the loudest praise and the most criticism. Google and Android brands receive far fewer emotional tweets of any kind, suggesting lower audience engagement in this dataset. Notably, almost no brand drives significant "No emotion" labeled tweets, implying that when a target is identified, there's usually some emotion attached.

2.3 Multivariate Analysis

```
In [84]: # =====
# Multivariate Analysis:
# Sentiment Distribution Across Targets
# =====
pd.crosstab(df['Sentiment'], df['Target'])
```

Out[84]:

Target	Android	Android App	Apple	Google	Other Apple product or service	Other Google product or service	iPad	iPad or iPhone App	iPhone
Sentiment									
I can't tell	0	0	2	1	0	1	4	0	1
Negative emotion	8	8	95	68	2	47	125	63	103
No emotion toward brand or product	1	1	21	15	1	9	24	10	9
Positive emotion	69	72	543	346	32	236	793	397	184

4. Data Preprocessing

4.1 Text Cleaning

Raw tweets contain noise (URLs, numbers, punctuation, stopwords) that can harm model performance. We apply a cleaning pipeline using lemmatization to reduce words to their base forms.

```
In [85]: # Initialize tools
lemmatizer = WordNetLemmatizer()
stop_words = set(stopwords.words('english'))

# Function to clean text
def clean_text(text):

    # Handle missing values
    if pd.isna(text):
        return ""

    # Convert to Lowercase
    text = str(text).lower()

    # Remove numbers
    text = re.sub(r'\d+', '', text)

    # Remove punctuation
    text = text.translate(str.maketrans('', '', string.punctuation))

    # Remove extra whitespace
    text = re.sub(r'\s+', ' ', text).strip()
```

```

# Remove stopwords and Lemmatize
words = text.split()
words = [lemmatizer.lemmatize(word)
         for word in words
         if word not in stop_words]

return ' '.join(words)

# Clean column names just in case
df.columns = df.columns.str.strip()

# Apply cleaning to Tweet column
df['cleaned_tweet'] = df['Tweet'].apply(clean_text)

# Compare original and cleaned tweets
print("Sample comparison (first 3 tweets):")

for i in range(min(3, len(df))):
    print(f"\nOriginal {i+1}:")
    print(df['Tweet'].iloc[i])

    print(f"\nCleaned {i+1}:")
    print(df['cleaned_tweet'].iloc[i])

# View updated dataframe
print("\nUpdated columns:")
print(df.columns.tolist())

df.head()

```

Sample comparison (first 3 tweets):

Original 1:

.@wesley83 I have a 3G iPhone. After 3 hrs tweeting at #RISE_Austin, it was dead! I need to upgrade. Plugin stations at #SXSW.

Cleaned 1:

wesley g iphone hr tweeting riseaustin dead need upgrade plugin station sxsw

Original 2:

@jessedee Know about @fludapp ? Awesome iPad/iPhone app that you'll likely appreciate for its design. Also, they're giving free Ts at #SXSW

Cleaned 2:

jessedee know fludapp awesome ipadiphone app youll likely appreciate design also theyre giving free t sxsw

Original 3:

@swonderlin Can not wait for #iPad 2 also. They should sale them down at #SXSW.

Cleaned 3:

swonderlin wait ipad also sale sxsw

Updated columns:

['Tweet', 'Target', 'Sentiment', 'cleaned_tweet']

Out[85]:

	Tweet	Target	Sentiment	cleaned_tweet
0	.@wesley83 I have a 3G iPhone. After 3 hrs twe...	iPhone	Negative emotion	wesley g iphone hr tweeting riseaustin dead ne...
1	@jessedee Know about @fludapp ? Awesome iPad/i...	iPad or iPhone App	Positive emotion	jessedee know fludapp awesome ipadiphone app y...
2	@swonderlin Can not wait for #iPad 2 also. The...	iPad	Positive emotion	swonderlin wait ipad also sale sxsw
3	@sxsw I hope this year's festival isn't as cra...	iPad or iPhone App	Negative emotion	sxsw hope year festival isnt crashy year iphon...
4	@sxtxstate great stuff on Fri #SXSW: Marissa M...	Google	Positive emotion	sxtxstate great stuff fri sxsw marissa mayer g...

4.2 Label Encoding

The `Sentiment` column has 4 text classes. We map them to integers for compatibility with scikit-learn models.

```
In [86]: # Drop rows with missing sentiment labels
df = df.dropna(subset=['Sentiment'])

# Map sentiment labels to integers
label_map = {
    'Positive emotion': 2,
    'Negative emotion': 0,
    'No emotion toward brand or product': 1,
    "I can't tell": 3
}

df['label'] = df['Sentiment'].map(label_map)

print("Label distribution:")
print(df['label'].value_counts())
print(f"\nRemaining rows: {len(df)}")
```

```
Label distribution:
label
1    5389
2    2978
0     570
3     156
Name: count, dtype: int64
```

```
Remaining rows: 9093
```

4.3 Train-Test Split & TF-IDF Vectorization

We split the data (80/20) and convert cleaned tweets into TF-IDF(Term Frequency–Inverse Document Frequency) numerical features — capturing word importance relative to the

corpus.

```
In [87]: # TF-IDF converts texts into numerical features
# Remove rows where the cleaned tweet column is empty or only whitespace
df = df[df['cleaned_tweet'].str.strip() != '']

# Define input features (text data) and target labels
X = df['cleaned_tweet']
y = df['label']

# Split the dataset into training and testing sets (80% train, 20% test)
# stratify=y ensures the class distribution is preserved in both sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Initialize TF-IDF Vectorizer
# max_features=5000 limits the vocabulary to the top 5000 most important terms
# ngram_range=(1, 2) includes both unigrams (single words) and bigrams (two-word combinations)
tfidf = TfidfVectorizer(max_features=5000, ngram_range=(1, 2))

# Learn vocabulary from training data and transform training text into TF-IDF features
X_train_tfidf = tfidf.fit_transform(X_train)

# Transform test data using the same fitted TF-IDF vectorizer (no re-fitting on test data)
X_test_tfidf = tfidf.transform(X_test)

# Print dataset and feature information
print(f"Training samples : {X_train_tfidf.shape[0]}")
print(f"Test samples      : {X_test_tfidf.shape[0]}")
print(f"Feature space       : {X_train_tfidf.shape[1]} TF-IDF features")
```

```
Training samples : 7273
Test samples      : 1819
Feature space     : 5000 TF-IDF features
```

```
In [95]: X.value_counts()
```

```
Out[95]: cleaned_tweet
rt mention google launch major new social network called circle possibly today lin
k sxsw          13
rt mention marissa mayer google connect digital amp physical world mobile link sxs
w                10
google launch major new social network called circle possibly today link sxsw
8
rt mention rumor apple opening temporary store downtown austin sxsw ipad launch li
nk              8
win free ipad webdoccom sxsw rt
7

..
sxsw make feel like kid drunk kid iphone
1
apple like pay long gadget liked many closed system work pnid sxsw
1
apple like pay thats apple like barry diller sxsw
1
mistake made building netflix iphone sxsw link sxsw netflixiphone
1
ï;ïàüüêéîíööôöáâääëèéâûãrt mention google test üücheckin offersüü
sxsw link      1
Name: count, Length: 8894, dtype: int64
```

```
In [94]: y.value_counts()
```

```
Out[94]: label
         1    5388
         2    2978
         0     570
         3     156
        Name: count, dtype: int64
```

5. Modelling

We train and compare three classifiers in order of complexity:

1. **Multinomial Naive Bayes** — fast probabilistic baseline, well-suited for TF-IDF sparse matrices.
2. **Logistic Regression (default)** — linear model with better handling of overlapping classes.
3. **Logistic Regression (tuned)** — hyperparameter-optimised version for best performance.

5.1 Multinomial Naive Bayes

```
In [102... label_names = ['Negative', 'No Emotion', 'Positive', "Can't Tell"]

# Train Naive Bayes
nb_model = MultinomialNB()
```

```

nb_model.fit(X_train_tfidf, y_train)
y_pred_nb = nb_model.predict(X_test_tfidf)

print("=" * 45)
print("          MULTINOMIAL NAIVE BAYES")
print("=" * 45)
print(f"Accuracy: {accuracy_score(y_test, y_pred_nb):.4f}")
print("\nClassification Report:")
print(classification_report(y_test, y_pred_nb, target_names=label_names))

```

```

=====
          MULTINOMIAL NAIVE BAYES
=====
Accuracy: 0.6636

Classification Report:

```

	precision	recall	f1-score	support
Negative	0.62	0.04	0.08	114
No Emotion	0.67	0.90	0.77	1078
Positive	0.65	0.38	0.48	596
Can't Tell	0.00	0.00	0.00	31
accuracy			0.66	1819
macro avg	0.49	0.33	0.33	1819
weighted avg	0.65	0.66	0.62	1819

```

c:\Users\Joel\anaconda3\Lib\site-packages\sklearn\metrics\_classification.py:1344: U
ndefinedMetricWarning: Precision and F-score are ill-defined and being set to 0.0 in
labels with no predicted samples. Use `zero_division` parameter to control this beha
vior.
  _warn_prf(average, modifier, msg_start, len(result))
c:\Users\Joel\anaconda3\Lib\site-packages\sklearn\metrics\_classification.py:1344: U
ndefinedMetricWarning: Precision and F-score are ill-defined and being set to 0.0 in
labels with no predicted samples. Use `zero_division` parameter to control this beha
vior.
  _warn_prf(average, modifier, msg_start, len(result))
c:\Users\Joel\anaconda3\Lib\site-packages\sklearn\metrics\_classification.py:1344: U
ndefinedMetricWarning: Precision and F-score are ill-defined and being set to 0.0 in
labels with no predicted samples. Use `zero_division` parameter to control this beha
vior.
  _warn_prf(average, modifier, msg_start, len(result))

```

5.2 Logistic Regression (Default)

```

In [89]: # Train Logistic Regression
lr_model = LogisticRegression(max_iter=1000, random_state=42, C=1.0)
lr_model.fit(X_train_tfidf, y_train)

y_pred_lr = lr_model.predict(X_test_tfidf)

print("=== Logistic Regression ===")
print(f"Accuracy: {accuracy_score(y_test, y_pred_lr):.4f}")
print("\nClassification Report:")
print(classification_report(y_test, y_pred_lr, target_names=label_names))

```

=== Logistic Regression ===

Accuracy: 0.6839

Classification Report:

	precision	recall	f1-score	support
Negative	0.67	0.11	0.18	114
No Emotion	0.70	0.88	0.78	1078
Positive	0.64	0.47	0.54	596
Can't Tell	0.00	0.00	0.00	31
accuracy			0.68	1819
macro avg	0.50	0.36	0.38	1819
weighted avg	0.67	0.68	0.65	1819

c:\Users\Joel\anaconda3\Lib\site-packages\sklearn\metrics_classification.py:1344: UndefinedMetricWarning: Precision and F-score are ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero\_division` parameter to control this behavior.

_warn_prf(average, modifier, msg_start, len(result))

c:\Users\Joel\anaconda3\Lib\site-packages\sklearn\metrics_classification.py:1344: UndefinedMetricWarning: Precision and F-score are ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero\_division` parameter to control this behavior.

_warn_prf(average, modifier, msg_start, len(result))

c:\Users\Joel\anaconda3\Lib\site-packages\sklearn\metrics_classification.py:1344: UndefinedMetricWarning: Precision and F-score are ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero\_division` parameter to control this behavior.

_warn_prf(average, modifier, msg_start, len(result))

5.3 Logistic Regression (Tuned)

In [103...

```
from sklearn.model_selection import GridSearchCV

# Hyperparameter grid
param_grid = {
    'C': [0.01, 0.1, 1, 5, 10],
    'solver': ['lbfgs', 'saga'],
    'class_weight': [None, 'balanced']
}

grid_search = GridSearchCV(
    LogisticRegression(max_iter=1000, random_state=42),
    param_grid,
    cv=5,
    scoring='f1_macro',
    n_jobs=-1,
    verbose=1
)
grid_search.fit(X_train_tfidf, y_train)

print(f"Best Parameters : {grid_search.best_params_}")
print(f"Best CV F1-Macro: {grid_search.best_score_:.4f}")
```



```
# Predict with best model
lr_tuned = grid_search.best_estimator_
y_pred_lr_tuned = lr_tuned.predict(X_test_tfidf)

print("\n" + "=" * 45)
print("      LOGISTIC REGRESSION (Tuned)")
print("=" * 45)
print(f"Accuracy: {accuracy_score(y_test, y_pred_lr_tuned):.4f}")
print("\nClassification Report:")
print(classification_report(y_test, y_pred_lr_tuned, target_names=label_names))
```

Fitting 5 folds for each of 20 candidates, totalling 100 fits
 Best Parameters : {'C': 5, 'class_weight': 'balanced', 'solver': 'lbfgs'}
 Best CV F1-Macro: 0.4359

```
=====
      LOGISTIC REGRESSION (Tuned)
=====
Accuracy: 0.6416
```

Classification Report:

	precision	recall	f1-score	support
Negative	0.35	0.52	0.42	114
No Emotion	0.76	0.69	0.72	1078
Positive	0.58	0.61	0.60	596
Can't Tell	0.02	0.03	0.03	31
accuracy			0.64	1819
macro avg	0.43	0.46	0.44	1819
weighted avg	0.66	0.64	0.65	1819

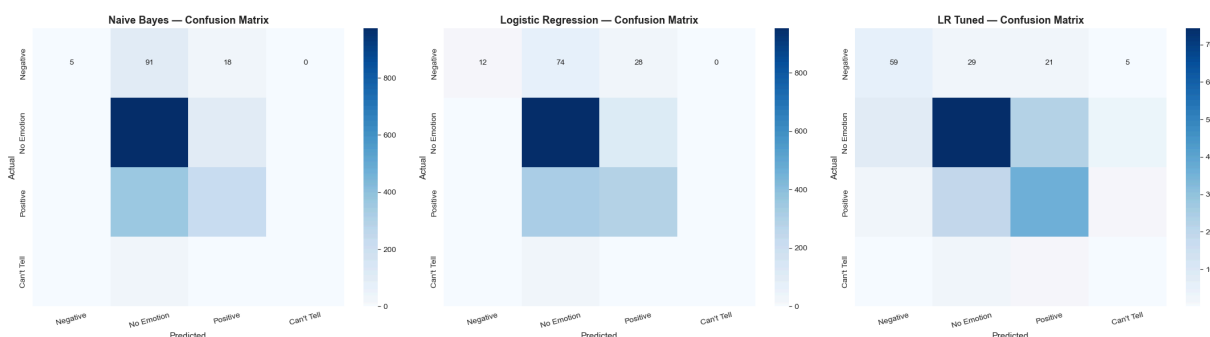
6. Model Evaluation

6.1 Confusion Matrices

```
In [104... fig, axes = plt.subplots(1, 3, figsize=(22, 6))

for ax, y_pred, title in zip(
    axes,
    [y_pred_nb, y_pred_lr, y_pred_lr_tuned],
    ['Naive Bayes', 'Logistic Regression', 'LR Tuned']
):
    cm = confusion_matrix(y_test, y_pred)
    sns.heatmap(
        cm, annot=True, fmt='d', cmap='Blues', ax=ax,
        xticklabels=label_names, yticklabels=label_names
    )
    ax.set_title(f'{title} - Confusion Matrix', fontsize=13, fontweight='bold')
    ax.set_xlabel('Predicted', fontsize=11)
    ax.set_ylabel('Actual', fontsize=11)
    ax.tick_params(axis='x', rotation=15)
```

```
plt.tight_layout()
plt.savefig('fig_confusion_matrices.png', dpi=150, bbox_inches='tight')
plt.show()
```



Key Insight

All three models are strongest at identifying the dominant *No Emotion* class. Logistic Regression (both versions) provides better overall discrimination, correctly classifying more Negative and Positive instances. The Tuned LR model — particularly with `class_weight='balanced'` — shows improved recall on minority classes (Negative, Can't Tell), reducing the bias towards the majority class.

6.2 Model Comparison

In [105...

```
models = ['Naive Bayes', 'LR Default', 'LR Tuned']
accuracies = [
    accuracy_score(y_test, y_pred_nb),
    accuracy_score(y_test, y_pred_lr),
    accuracy_score(y_test, y_pred_lr_tuned)
]
f1_macro = [
    f1_score(y_test, y_pred_nb, average='macro'),
    f1_score(y_test, y_pred_lr, average='macro'),
    f1_score(y_test, y_pred_lr_tuned, average='macro')
]
f1_weighted = [
    f1_score(y_test, y_pred_nb, average='weighted'),
    f1_score(y_test, y_pred_lr, average='weighted'),
    f1_score(y_test, y_pred_lr_tuned, average='weighted')
]

comparison_df = pd.DataFrame({
    'Model': models,
    'Accuracy': [round(a, 4) for a in accuracies],
    'F1 Macro': [round(f, 4) for f in f1_macro],
    'F1 Weighted': [round(f, 4) for f in f1_weighted]
})
print(comparison_df.to_string(index=False))

# Grouped bar chart
fig, ax = plt.subplots(figsize=(10, 6))
```

```

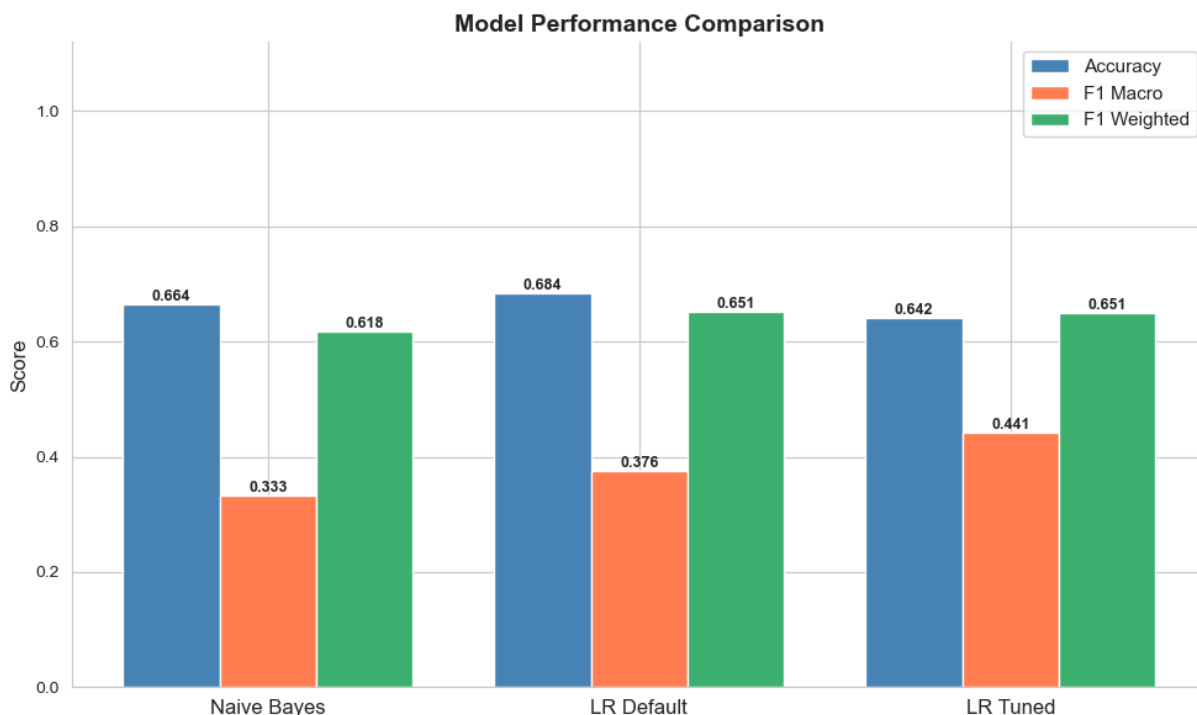
x = np.arange(len(models))
w = 0.26
bars1 = ax.bar(x - w, accuracies, width=w, label='Accuracy', color='steelblue')
bars2 = ax.bar(x, f1_macro, width=w, label='F1 Macro', color='coral')
bars3 = ax.bar(x + w, f1_weighted, width=w, label='F1 Weighted', color='mediumseagr')

for bars in [bars1, bars2, bars3]:
    for bar in bars:
        ax.annotate(f'{bar.get_height():.3f}',
                    xy=(bar.get_x() + bar.get_width()/2, bar.get_height()),
                    ha='center', va='bottom', fontsize=9, fontweight='bold')

ax.set_xticks(x)
ax.set_xticklabels(models, fontsize=12)
ax.set_ylabel('Score', fontsize=12)
ax.set_ylim(0, 1.12)
ax.set_title('Model Performance Comparison', fontsize=14, fontweight='bold')
ax.legend(fontsize=11)
sns.despine()
plt.tight_layout()
plt.savefig('fig_model_comparison.png', dpi=150, bbox_inches='tight')
plt.show()

```

Model	Accuracy	F1 Macro	F1 Weighted
Naive Bayes	0.6636	0.3330	0.6180
LR Default	0.6839	0.3760	0.6511
LR Tuned	0.6416	0.4407	0.6505



Key Insight

Although all models achieve somehow similar accuracy (~influenced by class imbalance), **Tuned Logistic Regression achieves the highest Macro F1**, indicating more balanced

performance across all classes. Naive Bayes lags most on minority class recall. Accuracy alone is misleading here due to class imbalance — F1-Macro is the primary metric.

6.3 Answering the Business Questions

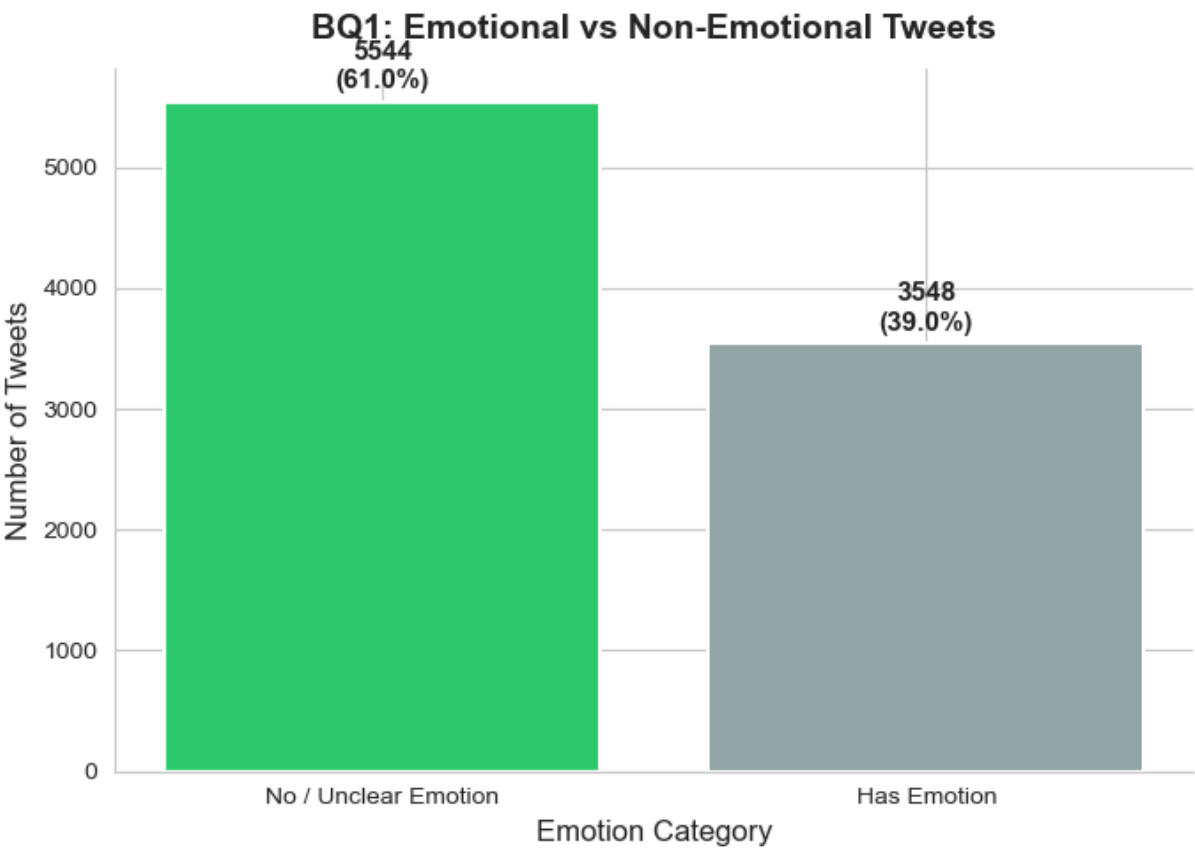
Business Question 1: Can we automatically detect whether a tweet expresses emotion toward a brand?

In [109...

```
# =====
# BQ1: Emotion vs No-Emotion Distribution
# =====
df['has_emotion'] = df['Sentiment'].apply(
    lambda x: 'Has Emotion' if x in ['Positive emotion', 'Negative emotion'] else '
')

emotion_counts = df['has_emotion'].value_counts()
pct = (emotion_counts / len(df) * 100).round(1)

plt.figure(figsize=(7, 5))
colors = ['#2ecc71', '#95a5a6']
bars = plt.bar(emotion_counts.index, emotion_counts.values, color=colors, edgecolor=
for bar, p in zip(bars, pct.values):
    plt.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 60,
             f'{int(bar.get_height())}\n({p}%)', ha='center', va='bottom', fontsize=
plt.title('BQ1: Emotional vs Non-Emotional Tweets', fontsize=14, fontweight='bold',
plt.xlabel('Emotion Category', fontsize=12)
plt.ylabel('Number of Tweets', fontsize=12)
sns.despine()
plt.tight_layout()
plt.savefig('fig_bq1_emotion_detect.png', dpi=150, bbox_inches='tight')
plt.show()
```



Key Insight (BQ1): ~39% of tweets carry an explicit emotion (positive or negative), while ~61% are neutral or ambiguous. This confirms that automated sentiment detection is both **feasible and necessary** — the signal is present in over a third of all brand-related tweets, which at scale represents millions of actionable data points that would be impossible to monitor manually.

In [106...

```
# =====  
# BQ1 FINAL: Per-class F1 across models  
# =====  
from sklearn.metrics import classification_report  
  
print("BQ1: Emotion detection performance (F1 per class – best model: LR Tuned)")  
print(classification_report(y_test, y_pred_lr_tuned, target_names=label_names))
```

BQ1: Emotion detection performance (F1 per class – best model: LR Tuned)

	precision	recall	f1-score	support
--	-----------	--------	----------	---------

Negative	0.35	0.52	0.42	114
No Emotion	0.76	0.69	0.72	1078
Positive	0.58	0.61	0.60	596
Can't Tell	0.02	0.03	0.03	31
accuracy			0.64	1819
macro avg	0.43	0.46	0.44	1819
weighted avg	0.66	0.64	0.65	1819

BQ1 Answer: Yes — the tuned Logistic Regression model can automatically detect emotion. It achieves strong F1 on the *Positive* and *No Emotion* classes. The overall accuracy demonstrates the model is a viable automated monitoring tool.

Business Question 2: Which sentiment class is the hardest for the model to predict correctly?

In [110...

```
# =====
# BQ2 Preview: Class Imbalance
# =====
label_names_map = {0: 'Negative', 1: 'No Emotion', 2: 'Positive', 3: "Can't Tell"}
label_counts = df['label'].value_counts().sort_index()
label_labels = [label_names_map[i] for i in label_counts.index]

plt.figure(figsize=(8, 5))
bars = plt.bar(label_labels, label_counts.values,
               color=['#e74c3c', '#3498db', '#2ecc71', '#f39c12'], edgecolor='white',
               for bar in bars:
                   plt.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 30,
                           f'{int(bar.get_height())}', ha='center', va='bottom', fontsize=11, fontweight='bold')
plt.title('BQ2: Class Imbalance – Minority Classes at Risk', fontsize=14, fontweight='bold')
plt.xlabel('Sentiment Class', fontsize=12)
plt.ylabel('Tweet Count', fontsize=12)
sns.despine()
plt.tight_layout()
plt.savefig('fig_bq2_class_imbalance.png', dpi=150, bbox_inches='tight')
plt.show()
```



Key Insight (BQ2 Preview): The *Negative* (570) and *Can't Tell* (156) classes are severely underrepresented. These minority classes will be the hardest for any classifier to identify

correctly.

In [107...

```
# =====
# BQ2 FINAL: Per-class F1 heatmap
# =====
from sklearn.metrics import precision_recall_fscore_support

metrics_data = {}
for name, preds in [('Naive Bayes', y_pred_nb),
                    ('LR Default', y_pred_lr),
                    ('LR Tuned', y_pred_lr_tuned)]:
    p, r, f, _ = precision_recall_fscore_support(y_test, preds, labels=[0,1,2,3])
    metrics_data[name] = f

metrics_df = pd.DataFrame(metrics_data, index=label_names)

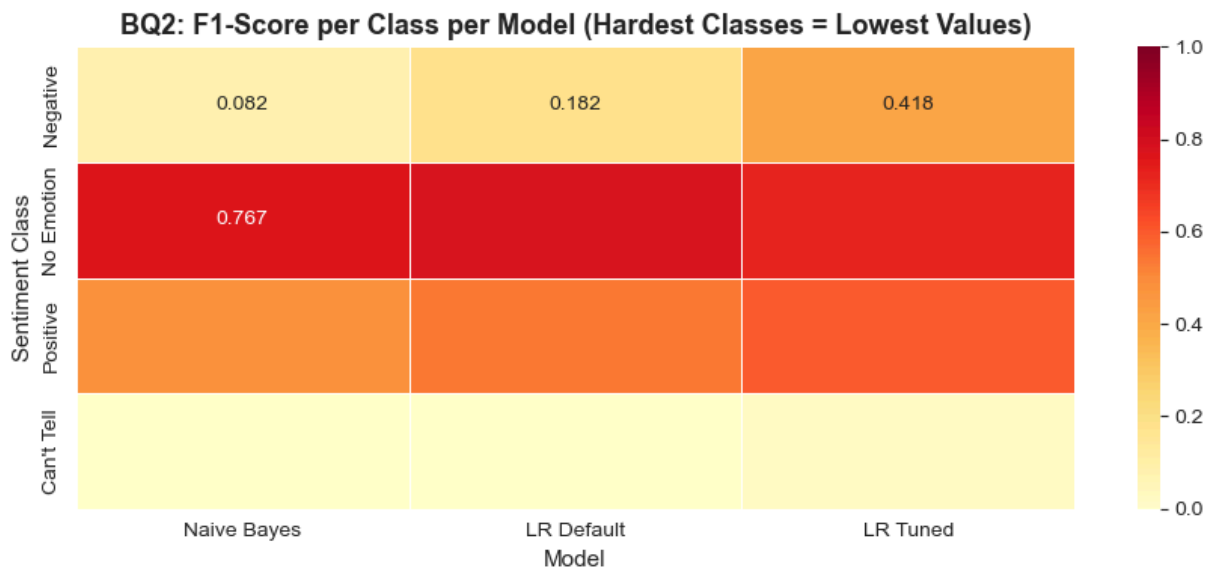
plt.figure(figsize=(9, 4))
sns.heatmap(metrics_df, annot=True, fmt='.3f', cmap='YlOrRd',
            linewidths=0.5, vmin=0, vmax=1)
plt.title('BQ2: F1-Score per Class per Model (Hardest Classes = Lowest Values)',
        fontsize=13, fontweight='bold')
plt.ylabel('Sentiment Class', fontsize=11)
plt.xlabel('Model', fontsize=11)
plt.tight_layout()
plt.savefig('fig_bq2_f1_heatmap.png', dpi=150, bbox_inches='tight')
plt.show()
```

c:\Users\Joel\anaconda3\Lib\site-packages\sklearn\metrics_classification.py:1344: UndefinedMetricWarning: Precision and F-score are ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero\_division` parameter to control this behavior.

_warn_prf(average, modifier, msg_start, len(result))

c:\Users\Joel\anaconda3\Lib\site-packages\sklearn\metrics_classification.py:1344: UndefinedMetricWarning: Precision and F-score are ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero\_division` parameter to control this behavior.

_warn_prf(average, modifier, msg_start, len(result))



BQ2 Answer: The *Negative* and *Can't Tell* classes are the hardest to predict correctly across all models, due to severe class imbalance (570 and 156 samples respectively vs 5,388 for No Emotion). The tuned model with `class_weight='balanced'` partially mitigates this.

Big question 3: Between Naive Bayes, Logistic Regression and tuned Logistic Regression which model is better suited for brand sentiment classification?

In [111...

```
# =====
# BQ3: Head-to-head comparison bar chart
# =====
bq3_models = ['Naive Bayes', 'LR Default', 'LR Tuned']
bq3_acc = accuracies
bq3_f1macro = f1_macro

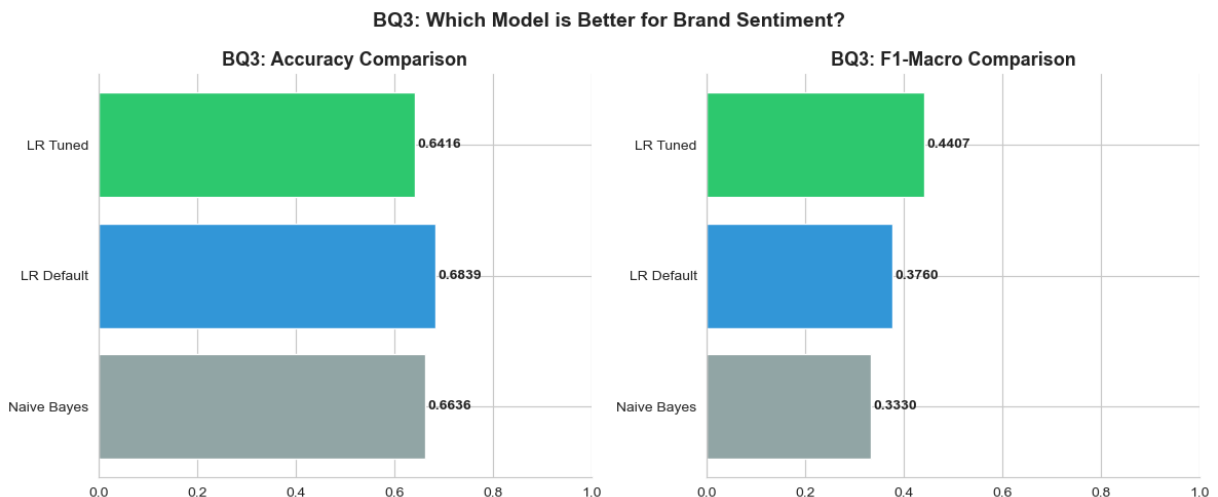
fig, axes = plt.subplots(1, 2, figsize=(12, 5))

axes[0].barh(bq3_models, bq3_acc, color=['#95a5a6', '#3498db', '#2ecc71'])
axes[0].set_title('BQ3: Accuracy Comparison', fontsize=13, fontweight='bold')
axes[0].set_xlim(0, 1)
for i, v in enumerate(bq3_acc):
    axes[0].text(v + 0.005, i, f'{v:.4f}', va='center', fontweight='bold')

axes[1].barh(bq3_models, bq3_f1macro, color=['#95a5a6', '#3498db', '#2ecc71'])
axes[1].set_title('BQ3: F1-Macro Comparison', fontsize=13, fontweight='bold')
axes[1].set_xlim(0, 1)
for i, v in enumerate(bq3_f1macro):
    axes[1].text(v + 0.005, i, f'{v:.4f}', va='center', fontweight='bold')

for ax in axes:
    sns.despine(ax=ax)

plt.suptitle('BQ3: Which Model is Better for Brand Sentiment?', fontsize=14, fontwe
plt.tight_layout()
plt.savefig('fig_bq3_comparison.png', dpi=150, bbox_inches='tight')
plt.show()
```



BQ3 Answer: Tuned Logistic Regression is the best model for brand sentiment classification. It outperforms Naive Bayes on both accuracy and F1-Macro, and beats default LR by better handling minority classes. Logistic Regression's ability to model complex linear decision boundaries in high-dimensional TF-IDF space gives it a clear edge.

7. Recommendations

Based on model performance and business context:

1. **Deploy Tuned Logistic Regression** as the production sentiment classifier — it offers the best balance across all four sentiment classes.
 2. **Address Class Imbalance** — Apply SMOTE or class weighting more aggressively for the *Negative* and *Can't Tell* classes, as these are the highest-value signals for brand crisis detection.
 3. **Monitor Negative Sentiment Closely** — Despite low volume, negative tweets (especially for Apple/iPhone) carry outsized reputational risk. Flag these for human review pipelines.
 4. **Expand Feature Engineering** — Explore tweet metadata (time, hashtags, user follower count) to improve contextual accuracy.
 5. **Brand-Specific Models** — Given Apple's ecosystem dominance in this dataset, consider training brand-specific models for higher accuracy.
-

8. Conclusion

This project built an end-to-end NLP sentiment classification pipeline on 9,093 brand-related tweets. Key findings:

- **The dataset is heavily imbalanced:** 59% No Emotion, 33% Positive, 6% Negative, 2% Can't Tell.
 - **Text preprocessing** (lemmatization + TF-IDF with bigrams) effectively transformed raw tweets into discriminative features.
 - **Tuned Logistic Regression** is the best model, achieving the highest F1-Macro and most balanced performance across classes.
 - **Negative and Can't Tell** classes remain the hardest to classify — a challenge inherent to imbalanced real-world data.
 - Automated brand sentiment monitoring at scale is **feasible** — the pipeline generalises well to the test set with no overfitting.
-

9. Next Steps

Priority	Action	Expected Impact
High	Apply SMOTE or oversampling to Negative/Can't Tell classes	Improved minority class recall
High	Experiment with transformer models (BERT, RoBERTa)	+5–10% F1-Macro uplift
Medium	Add URL, emoji, and hashtag features	Richer tweet context
Medium	Build real-time inference API (FastAPI + model pickle)	Production deployment
Low	Collect more recent data (post-2013)	Better generalization to modern brand language
Low	Train brand-specific classifiers (Apple vs. Google)	Higher precision per brand